

CSC458H1: Computer Networks — Fall 2026

Programming Assignment 0: Warm-up

Due Date: Sep 25, 2026, 11:59 PM

Overview

Welcome to the CSC458 programming assignments. PA0 is a short warm-up whose goals are to verify that your virtual-machine environment works, give you practice running Python programs and automated tests from the Linux terminal, and introduce direct TCP socket programming through a tiny HTTP client. All later programming assignments use Python and Linux networking tools inside the same course VM. Make sure to check out the [Marking Scheme and Assignment Submission Policy](#) and [GenAI Policy](#) on Quercus for details on collaboration, code use, and use of generative AI. A0 is to be done *individually*.

Learning objectives

By the end of this assignment you should be able to:

- Use the course VM and command-line tools
- Write a basic HTTP GET request
- Use TCP sockets to send and receive data
- Understand that TCP is a byte stream, so data may arrive across multiple `recv()` calls

1 Environment Setup

The official execution and grading environment is the course Ubuntu 22.04 virtual machine. The set up instructions are available on [Quercus](#). You're expected to complete the assignment *inside the VM*, even if Python is also installed on your Windows/macOS host.

Then extract the PA0 starter package into your Linux home directory (not a VirtualBox shared folder), enter the repository, and run:

```
python3 tools/check_environment.py
```

The expected Python version in the course VM is Python 3.10 or newer. PA0 uses only the Python standard library; you do not need to install any Python package.

Run the public tests once before changing the code:

```
python3 -m unittest discover -s tests -v
```

The starter code imports successfully, but the functional tests should fail because `get_url()` is intentionally incomplete.

Important. The graders will execute your submission in the same VM environment. Do not add dependencies or require a different Python version.

2 A Simple HTTP GET Request

Before writing code, you will manually perform the same exchange your program will later implement. HTTP is an application-layer protocol that normally runs over a reliable byte stream such as TCP. In

this PA0 we intentionally use unencrypted HTTP on port 80 so that you can see the request and response directly.

2.1 Optional external-server exercise

Open <https://api.ipify.org/> in a browser and note the public IP address shown. The browser uses HTTPS. We will now send a similar request manually over plain HTTP. In the VM, run:

```
nc api.ipify.org 80
```

After the TCP connection is established, type exactly the following lines, pressing Enter after each one and pressing Enter once more after the `Connection` line:

```
GET / HTTP/1.1
Host: api.ipify.org
Connection: close
```

Conceptually, the bytes sent on the connection are:

```
GET / HTTP/1.1\r\n
Host: api.ipify.org\r\n
Connection: close\r\n
\r\n
```

Each HTTP header line ends with the two bytes CRLF (`\r\n`). The extra empty line terminates the header block.

A typical response begins with a status line and headers and ends with a short body containing your public IP address:

```
HTTP/1.1 200 OK
Content-Type: text/plain
Content-Length: 12
Connection: close
...

138.51.43.127
```

The exact headers, server address, and IP value can change.

Note. External Internet access is **not** part of the grading contract. If `api.ipify.org` is unreachable because of a firewall, VPN, DNS problem, or temporary service issue, continue with the local exercise below. All automated grading is local and deterministic.

2.2 Deterministic local exercise

Open a first terminal and run:

```
python3 tools/local_http_server.py --port 8080
```

Leave it running. In a second terminal, connect with Netcat:

```
nc 127.0.0.1 8080
```

Then type:

```
GET /hello?course=CSC458 HTTP/1.1
Host: 127.0.0.1:8080
Connection: close
```

The local server will return a small deterministic response and close the connection.

3 Writing `webget.py`

The main task is to complete a small Python program that performs the preceding exchange using the operating system's TCP socket interface.

Your implementation will:

1. construct an HTTP/1.1 GET request;
2. ask the operating system to establish a TCP connection to the requested host and port;
3. send the request bytes;
4. repeatedly receive response bytes until the peer closes its outgoing byte stream; and
5. return the complete response unchanged.

3.1 Starter repository

The PA0 repository contains:

- `src/webget.py`: the only Python source file you modify
- `writeups/pa0.md`: short required writeup
- `tests/test_webget.py`: public automated tests
- `tools/check_environment.py`: VM/environment check
- `tools/local_http_server.py`: deterministic local test server

Open `src/webget.py`. You will find the function:

```
def get_url(host: str, path: str,
            port: int = 80, timeout: float = 5.0) -> bytes:
    ...
```

Complete `get_url()` using Python's `socket` module.

3.2 Rules for the implementation

- You **must** use a TCP socket directly. Python's `socket.create_connection()` is allowed.
- Do **not** use a higher-level HTTP client such as `urllib`, `http.client`, `requests`, `aiohttp`, or `httpx`.
- `path` must begin with `/`. If it does not, raise `ValueError`.
- Send an HTTP/1.1 GET request containing at least the `Host` and `Connection` headers. Include `:port` in the `Host` header when using an HTTP port other than 80.
- End every request/header line with `\r\n`; end the complete header block with an additional blank line.
- Use a method such as `sendall()` that ensures the complete request is transmitted.
- A single call to `recv()` is **not** enough. TCP is a byte stream. Repeatedly receive until `recv()` returns `b""`.
- Return the complete response as `bytes`. Do not assume the body is UTF-8 text.

The supplied command-line wrapper will write those bytes to standard output.

3.3 Manual testing

With the supplied local server still running on port 8080:

```
python3 src/webget.py 127.0.0.1 /hello --port 8080
```

When that works, you may optionally try:

```
python3 src/webget.py api.ipify.org /
```

Again, the external test is experiential only and is not graded.

3.4 Automated tests

Run the automated tests: `python3 -m unittest discover -s tests -v`

The public test suite checks, among other things, correct request formatting, CRLF termination, the `Host` and `Connection` headers, query strings, and the requirement to keep reading when the server splits a response across many TCP receives. The tests use a loopback server and do not require Internet access.

4 Hints

1. Python strings and network bytes are different types. Construct the textual HTTP request and encode it to bytes before sending it.
2. `socket.create_connection((host, port), timeout=...)` handles DNS resolution and establishes a stream socket.
3. A `with` statement is a convenient way to make sure the socket is closed even if an exception occurs.
4. TCP does not preserve application-level message boundaries. The server may divide one HTTP response among any number of `recv()` calls.
5. Because your request includes `Connection: close`, EOF is sufficient for this warm-up client to identify the end of the response. You are *not* asked to implement general HTTP framing such as chunked transfer encoding.

5 Writeup

Complete `writeups/pa0.md`. In addition to identifying yourself and crediting any help/resources, answer the four mandatory questions about CRLF framing, repeated `recv()` calls, `Connection: close`, and why the program preserves the response as bytes.

6 Submission and Mark Breakdown

Submit the following two files to the MarkUs assignment for PA0:

- `src/webget.py`
- `writeups/pa0.md`

The graders use deterministic local tests in the official VM. They may use additional hidden test cases, including larger and binary response bodies. Do not hard-code the host, path, response, or public-test behavior.

The 10 marks are allocated as follows:

- (i) Environment/API/syntax and no prohibited HTTP library: 1 mark.
- (ii) Correct general implementation of `get_url()`: 8 marks.
- (iii) Complete and correct `pa0.md`: 1 mark.